

Year 3 Semester 2

NIT3171 : ICT Business Analytics
and Data Visualization

Assessment 2 : Individual Project

Diabetes Dataset Analysis and Prediction

Dulan Nithila Liyanarachchi

Table of Contents

Chapter 1 : Dataset Selection.....	1
Chapter 2 : Data Exploring and Preprocessing	2
Chapter 3 : Data Visualization & Analysis.....	5
Chapter 4 : Literature Review	9
Literature Review	9
Summary Table	11
Explanation of Gaps Selected in Summary Table	13
Chapter 5 : Research Questions	14
Chapter 6 : Modelling and Evaluation	15
Summary and Conclusion	25
References	28
Appendix.....	29

List of Figures

Figure 1 : Requesting Lecturer Approval for Dataset	1
Figure 2 : Installing and Importing the required Libraries	2
Figure 3 : Loading the diabetes dataset and displaying it.....	2
Figure 4 : Displaying the Data Types of the Dataset	2
Figure 5 : Finding The Missing Values Count in Each Column.....	3
Figure 6 : Displaying the Summary Statistics of The Dataset.....	3
Figure 7 : Median - based imputation for zeros.....	4
Figure 8 : Outliers Displaying	4
Figure 9 : Installing and Importing the required Libraries	5
Figure 10 : Histogram Distributions of All Variables	6
Figure 11 : Boxplot Distributions of All Variables.....	7
Figure 12 : Line Graphs of All Variables over Index.....	7
Figure 13 : Scatter plots vs Outcome of All Variables	8
Figure 14 : Correlation Matrix of All Variables.....	8
Figure 15 : Installing and Importing the required Libraries	17
Figure 16 : Splitting the Data into Training and Testing Data Sets	17
Figure 17 : Logistic Regression model.....	18
Figure 18 : Random Forest model.....	19
Figure 19 : Confusion Matrices	20
Figure 20 : Histogram Distributions Figure	29
Figure 21 : Boxplot Distributions Figure.....	29
Figure 22 : Line Graphs Figure.....	30
Figure 23 : Scatter plots Figure	30
Figure 24 : Correlation Matrix Figure.....	30
Figure 25 : Confusion Matrices Figure	31

List of Tables

Table 1 : Literature Review Summary Table	12
Table 2 : Performance Evaluation Metrics Comparison	23

List of Abbreviations

1. BMI - Body Mass Index
2. T2DM - Type 2 diabetes mellitus
3. XGBoost - eXtreme Gradient Boosting
4. AUC - Area under the Curve
5. USA - United State of America
6. ID - Identification or Identity Document
7. ML - Machine Learning
8. AF/ablation - Atrial Fibrillation Ablation
9. SVM - Support Vector Machine
10. ANN - Artificial Neural Network

Chapter 1 : Dataset Selection

For this study, a publicly available diabetes dataset from Kaggle was selected, as it provides reliable and well - structured data suitable for academic analysis. The dataset, titled “Diabetes Dataset” (<https://www.kaggle.com/mathchi/datasets/diabetes-data-set>), includes important health - related variables such as age, blood glucose levels, Body Mass Index (BMI), insulin levels, and other clinical measurements that are commonly used in diabetes research.

This dataset was chosen because it clearly represents real - world medical data and includes a labeled outcome indicating whether an individual has diabetes or not. This makes it useful for exploring relationships between different health factors and diabetes, as well as for building predictive models. Although information related to lifestyle factors such as diet and exercise is limited, the available clinical and demographic attributes are sufficient to carry out meaningful analysis and obtain reliable results.

Before starting the analysis, the selected dataset was shared with the lecturer through an email to request approval. After reviewing the dataset and its suitability for the study, the lecturer provided approval to proceed. This confirmation ensured that the dataset meets the required academic standards and is appropriate for use in the analysis.

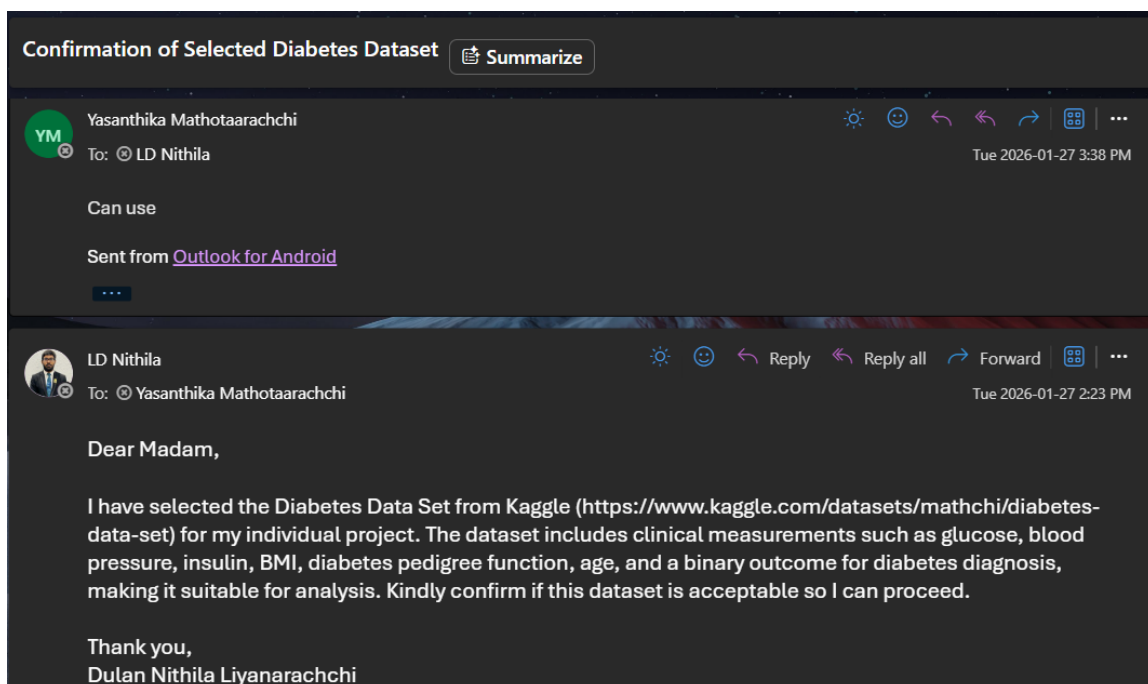


Figure 1 : Requesting Lecturer Approval for Dataset

Chapter 2 : Data Exploring and Preprocessing

```

1 #Installing and importing necessary libraries
2 %pip install pandas
3 %pip install numpy
4
5 # Importing necessary libraries
6 import pandas as pd
7 import numpy as np
✓ 36s
Python

Requirement already satisfied: pandas in c:\Users\Dulan Liyanarachchi\AppData\Local\Programs\Python\Python314\Lib\site-packages (3.0.0)
Requirement already satisfied: numpy>=2.3.3 in c:\Users\Dulan Liyanarachchi\AppData\Local\Programs\Python\Python314\Lib\site-packages (from pandas) (2.4.2)
Requirement already satisfied: python-dateutil>=2.8.2 in c:\Users\Dulan Liyanarachchi\AppData\Local\Programs\Python\Python314\Lib\site-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: tzdata in c:\Users\Dulan Liyanarachchi\AppData\Local\Programs\Python\Python314\Lib\site-packages (from pandas) (2025.3)
Requirement already satisfied: six>=1.5 in c:\Users\Dulan Liyanarachchi\AppData\Local\Programs\Python\Python314\Lib\site-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
Note: you may need to restart the kernel to use updated packages.
Requirement already satisfied: numpy in c:\Users\Dulan Liyanarachchi\AppData\Local\Programs\Python\Python314\Lib\site-packages (2.4.2)
Note: you may need to restart the kernel to use updated packages.

```

Figure 2 : Installing and Importing the required Libraries

- To do the preprocessing and data exploration I installed pandas, and numpy libraries and imported them which are clearly displayed in above figure.

```

1 # loading the diabetes dataset and displaying it(which will show the first and last few rows in details and others in dots)
2 df = pd.read_csv("diabetes.csv")
3 print
4 (
5 | df
6 )
✓ 0.0s
Python

```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	6	143	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1
...	...	...	...	...	...	...	...	...	...
763	10	101	76	48	180	32.9	0.171	63	0
764	2	122	70	27	0	36.8	0.340	27	0
765	5	121	72	23	112	26.2	0.245	30	0
766	1	126	60	0	0	30.1	0.349	47	1
767	1	93	70	31	0	30.4	0.315	23	0

768 rows × 9 columns

Figure 3 : Loading the diabetes dataset and displaying it

- As shown in the above figure The Dataset was loaded to a data frame named “df” using pd.read_csv function and displayed.
- Data set has 9 columns named Pregnancies, Glucose, BloodPressure, SkinThickness, Insulin, BMI, DiabetesPedigreeFunction, Age, Outcome and 768 rows

```

1 #data types
2 print(df.dtypes)
✓ 0.0s
Python

```

Pregnancies	int64
Glucose	int64
BloodPressure	int64
SkinThickness	int64
Insulin	int64
BMI	float64
DiabetesPedigreeFunction	float64
Age	int64
Outcome	int64
dtype: object	

Figure 4 : Displaying the Data Types of the Dataset

- Then I searched for the Data types in the Data set using df.dtypes function and displayed.
- There are no categorical variables in the dataset as shown in the above figure(Figure 4).

- So, all 9 Variables are numerical variables.

```
1 #missing values count for each column
2 print(df.isnull().sum())

✓ 0.0s Python
```

Variable	Count
Pregnancies	0
Glucose	0
BloodPressure	0
SkinThickness	0
Insulin	0
BMI	0
DiabetesPedigreeFunction	0
Age	0
Outcome	0

dtype: int64

Figure 5 : Finding The Missing Values Count in Each Column

- Next I searched for the missing values count in the dataset using isnull().sum function which was applied to the data frame df.
- From that I got a result a that there are no missing values in the data set which is shown in the above figure(Figure 5).

```
1 # Summary statistics
2 print
3 (
4     df.describe()
5 )

✓ 0.0s Python
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
count	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000
mean	3.845052	120.894531	69.105469	20.536458	79.799479	31.992578	0.471876	33.240885	0.348958
std	3.369578	31.972618	19.355807	15.952218	115.244002	7.884160	0.331329	11.760232	0.476951
min	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.078000	21.000000	0.000000
25%	1.000000	99.000000	62.000000	0.000000	0.000000	27.300000	0.243750	24.000000	0.000000
50%	3.000000	117.000000	72.000000	23.000000	30.500000	32.000000	0.372500	29.000000	0.000000
75%	6.000000	140.250000	80.000000	32.000000	127.250000	36.600000	0.626250	41.000000	1.000000
max	17.000000	199.000000	122.000000	99.000000	846.000000	67.100000	2.420000	81.000000	1.000000

Figure 6 : Displaying the Summary Statistics of The Dataset

- In Figure 6 is shows the summary statistics of the data set, which includes the count of values, mean, standard deviation, minimum value, maximum value, 1st Quarter Value(25%), Median/ 2nd Quarter Value(50%), 3rd Quarter Value(75%), and Maximum Value for all variables/columns of the dataset.

```
1 # Check zeros in each column except Pregnancies
2 print("Zeros in each column (excluding Pregnancies and Outcome)")
3 print((df == 0).sum().drop(['Pregnancies', 'Outcome']))
4
5 # Replace zeros with median, excluding Pregnancies and Outcome
6 df[['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI']] = df[['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI']].replace(0, np.nan)
7
8 # Fill Glucose, BP, and BMI with global median (small number of missing values)
9 for col in ['Glucose', 'BloodPressure', 'BMI']: df[col] = df[col].fillna(df[col].median())
10
11 # Fill Insulin and SkinThickness with grouped median (high number of missing values)
12 for col in ['Insulin', 'SkinThickness']: df[col] = df[col].fillna(df.groupby('Outcome')[col].transform('median'))
13
14 print("\nZeros in each column after replacement (excluding Pregnancies and Outcome)")
15 print((df == 0).sum().drop(['Pregnancies', 'Outcome']))

✓ 0.0s Python
```

Zeros in each column (excluding Pregnancies and Outcome)

Variable	Count
Glucose	5
BloodPressure	35
SkinThickness	227
Insulin	374
BMI	11
DiabetesPedigreeFunction	0
Age	0

dtype: int64

Zeros in each column after replacement (excluding Pregnancies and Outcome)

Variable	Count
Glucose	0
BloodPressure	0
SkinThickness	0
Insulin	0
BMI	0
DiabetesPedigreeFunction	0
Age	0

dtype: int64

Figure 7 : Median - based imputation for zeros

- As shown in the above figure, I handled missing values by first converting zero values to NaN, since measurements like Insulin or Blood Pressure cannot realistically be zero.
- For features with low missing values, such as BMI and Glucose, I filled them using the overall median.
- For variables with high missing values, like Insulin and SkinThickness, I used the median within each Outcome group (diabetic vs. non - diabetic). This helps keep the data realistic and meaningful.

```
1 # Outliers (IQR) + capping
2 num = df.select_dtypes(include=[np.number]).columns
3 q1, q3 = df[num].quantile(0.25), df[num].quantile(0.75)
4 iqr = q3 - q1
5 lb, ub = q1 - 1.5 * iqr, q3 + 1.5 * iqr
6
7 mask = (df[num] < lb) | (df[num] > ub)
8 print("Outlier count per column")
9 print(mask.sum().sort_values(ascending=False))
```

✓ 0.0s

Outlier count per column	
SkinThickness	87
Insulin	51
DiabetesPedigreeFunction	29
BloodPressure	14
Age	9
BMI	8
Pregnancies	4
Glucose	0
Outcome	0

dtype: int64

Figure 8 : Outliers Displaying

- At last, I searched for the outlier count for each variable which is clearly mentioned in the above figure(Figure 8), and I found many outliers in columns like SkinThickness, Insulin, and Diabetes Pedigree Function.
- I decided to keep them because extreme values often reflect medical conditions, such as high Insulin or genetic risk, and removing them could lose important information.
- Some outliers appeared due to replacing zeros with medians, so they aren't errors. Keeping these values helps maintain a realistic and diverse dataset.

Chapter 3 : Data Visualization & Analysis

```

1 #installing and importing necessary libraries
2 %pip install seaborn
3 %pip install matplotlib
4
5 # Importing necessary libraries
6 import seaborn as sns
7 import matplotlib.pyplot as plt
(11) ✓ 29s

-- Requirement already satisfied: seaborn in c:\Users\Dulan\Liyanarachchi\AppData\Local\Programs\Python\Python314\Lib\site-packages (0.13.2)
Requirement already satisfied: numpy<1.24.0,>=1.20 in c:\Users\Dulan\Liyanarachchi\AppData\Local\Programs\Python\Python314\Lib\site-packages (from seaborn) (2.4.2)
Requirement already satisfied: pandas>=1.2 in c:\Users\Dulan\Liyanarachchi\AppData\Local\Programs\Python\Python314\Lib\site-packages (from seaborn) (3.0.0)
Requirement already satisfied: matplotlib>=3.6.1,>=3.4 in c:\Users\Dulan\Liyanarachchi\AppData\Local\Programs\Python\Python314\Lib\site-packages (from seaborn) (3.10.0)
Requirement already satisfied: contourpy>=1.0.1 in c:\Users\Dulan\Liyanarachchi\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib>=3.6.1,>=3.4->seaborn) (1.3.3)
Requirement already satisfied: cycler>=0.10 in c:\Users\Dulan\Liyanarachchi\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib>=3.6.1,>=3.4->seaborn) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in c:\Users\Dulan\Liyanarachchi\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib>=3.6.1,>=3.4->seaborn) (4.61.1)
Requirement already satisfied: kiwisolver>=1.3.1 in c:\Users\Dulan\Liyanarachchi\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib>=3.6.1,>=3.4->seaborn) (1.4.9)
Requirement already satisfied: packaging>20.0 in c:\Users\Dulan\Liyanarachchi\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib>=3.6.1,>=3.4->seaborn) (26.0)
Requirement already satisfied: pillow>=8 in c:\Users\Dulan\Liyanarachchi\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib>=3.6.1,>=3.4->seaborn) (12.1.0)
Requirement already satisfied: pyparsing>=3 in c:\Users\Dulan\Liyanarachchi\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib>=3.6.1,>=3.4->seaborn) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in c:\Users\Dulan\Liyanarachchi\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib>=3.6.1,>=3.4->seaborn) (2.9.0.post0)
Requirement already satisfied: tzdata in c:\Users\Dulan\Liyanarachchi\AppData\Local\Programs\Python\Python314\Lib\site-packages (from pandas>=1.2->seaborn) (2025.3)
Requirement already satisfied: six>=1.5 in c:\Users\Dulan\Liyanarachchi\AppData\Local\Programs\Python\Python314\Lib\site-packages (from python-dateutil>=2.7->matplotlib>=3.6.1,>=3.4->seaborn) (1.17.0)
Note: you may need to restart the kernel to use updated packages.
Requirement already satisfied: matplotlib in c:\Users\Dulan\Liyanarachchi\AppData\Local\Programs\Python\Python314\Lib\site-packages (3.10.0)
Requirement already satisfied: contourpy>=1.0.1 in c:\Users\Dulan\Liyanarachchi\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in c:\Users\Dulan\Liyanarachchi\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in c:\Users\Dulan\Liyanarachchi\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib) (4.61.1)
Requirement already satisfied: kiwisolver>=1.3.1 in c:\Users\Dulan\Liyanarachchi\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib) (1.4.9)
Requirement already satisfied: numpy>=1.23 in c:\Users\Dulan\Liyanarachchi\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib) (2.4.2)
Requirement already satisfied: packaging>20.0 in c:\Users\Dulan\Liyanarachchi\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib) (26.0)
Requirement already satisfied: pillow>=8 in c:\Users\Dulan\Liyanarachchi\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib) (12.1.0)
Requirement already satisfied: pyparsing>=3 in c:\Users\Dulan\Liyanarachchi\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in c:\Users\Dulan\Liyanarachchi\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in c:\Users\Dulan\Liyanarachchi\AppData\Local\Programs\Python\Python314\Lib\site-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)
Note: you may need to restart the kernel to use updated packages.

```

Figure 9 : Installing and Importing the required Libraries

- To do the data visualization and analysis I installed matplotlib, and seaborn libraries and imported them which are clearly displayed in above figure.

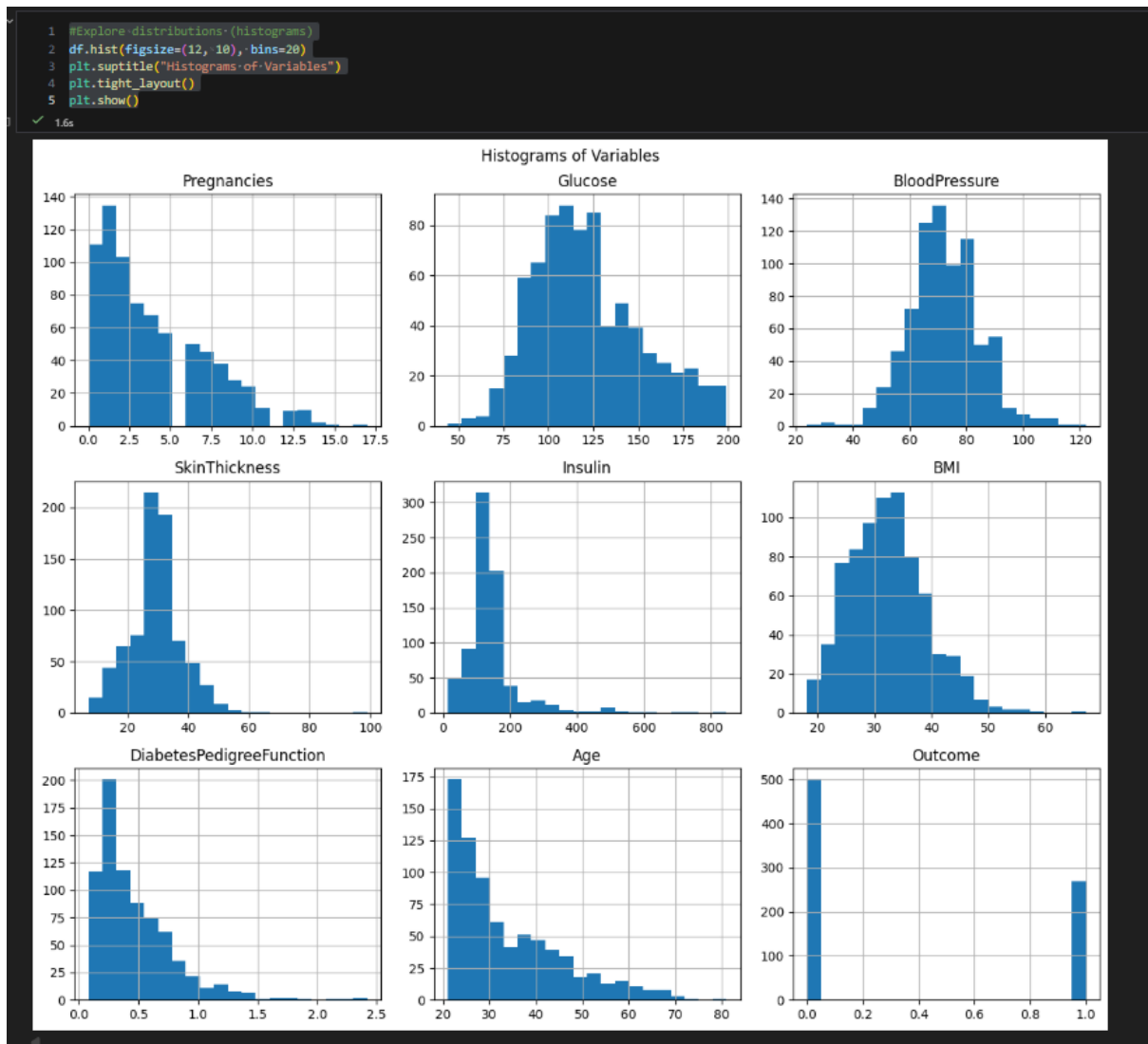


Figure 10 : Histogram Distributions of All Variables

- Next, to analyze the variation in all variables, I created separate histograms for each variable to better understand the distribution and overall pattern of the data. For Further Clarification I have attached only the figure of the Histograms separately in the Appendix Section.

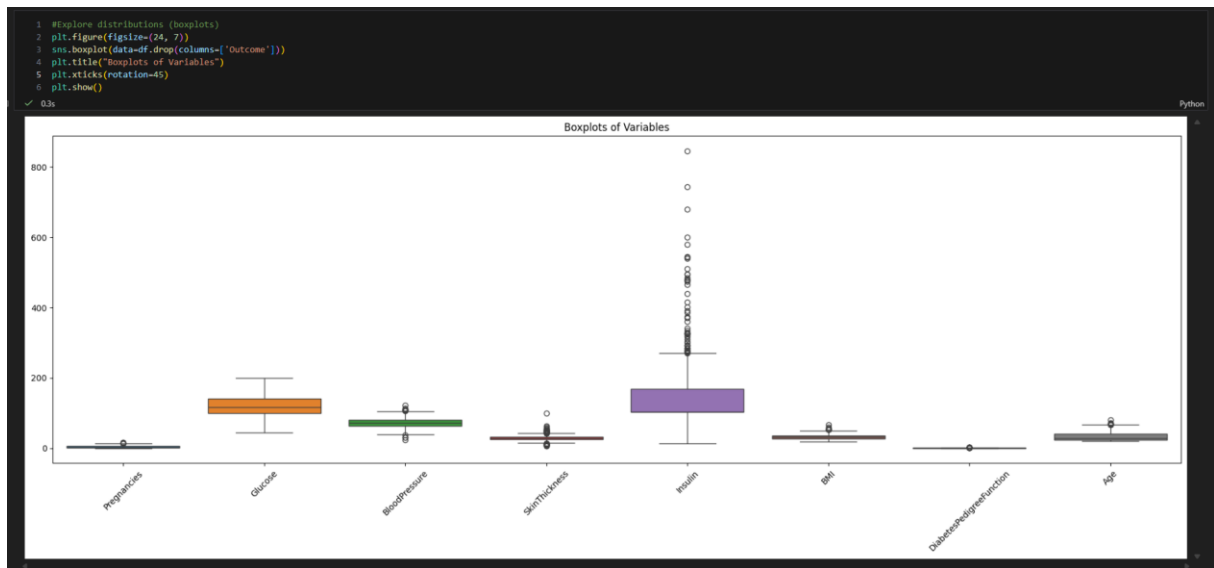


Figure 11 : Boxplot Distributions of All Variables

- Then I generated boxplots for each variable to clearly examine the spread of the data and outliers. For Further Clarification I have attached the figure of the Boxplots in the Appendix Section. For Further Clarification I have attached only the figure of the Boxplots separately in the Appendix Section.

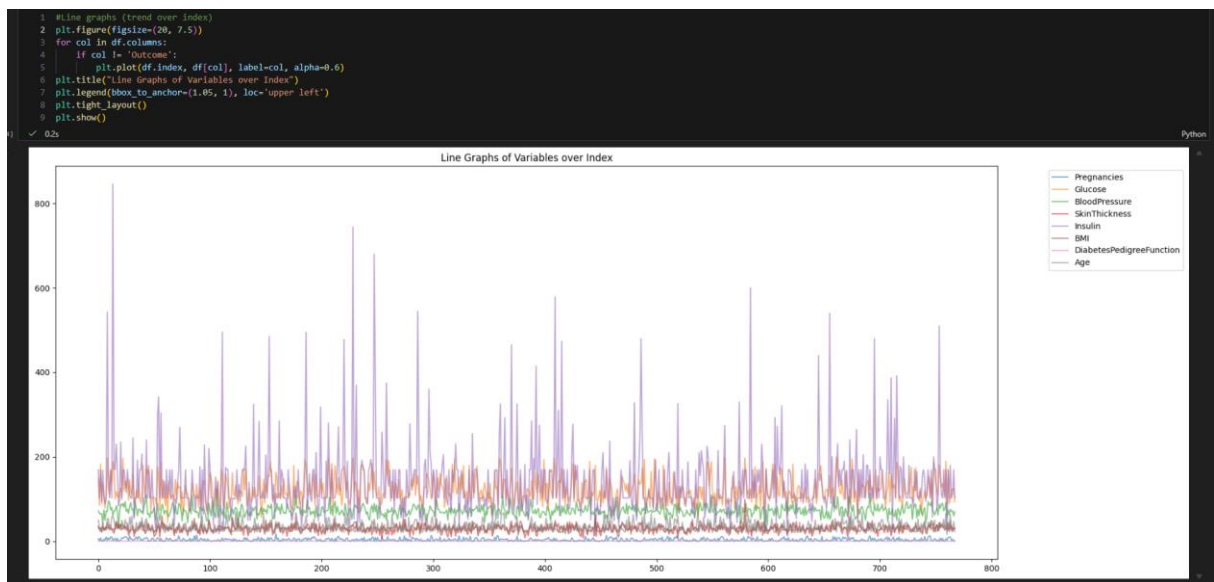


Figure 12 : Line Graphs of All Variables over Index

- Next, I plotted line graphs of each variable against the index to observe how the values change across observations and to identify any visible trends or patterns over time. For Further Clarification I have attached only the figure of the Line Graphs separately in the Appendix Section.

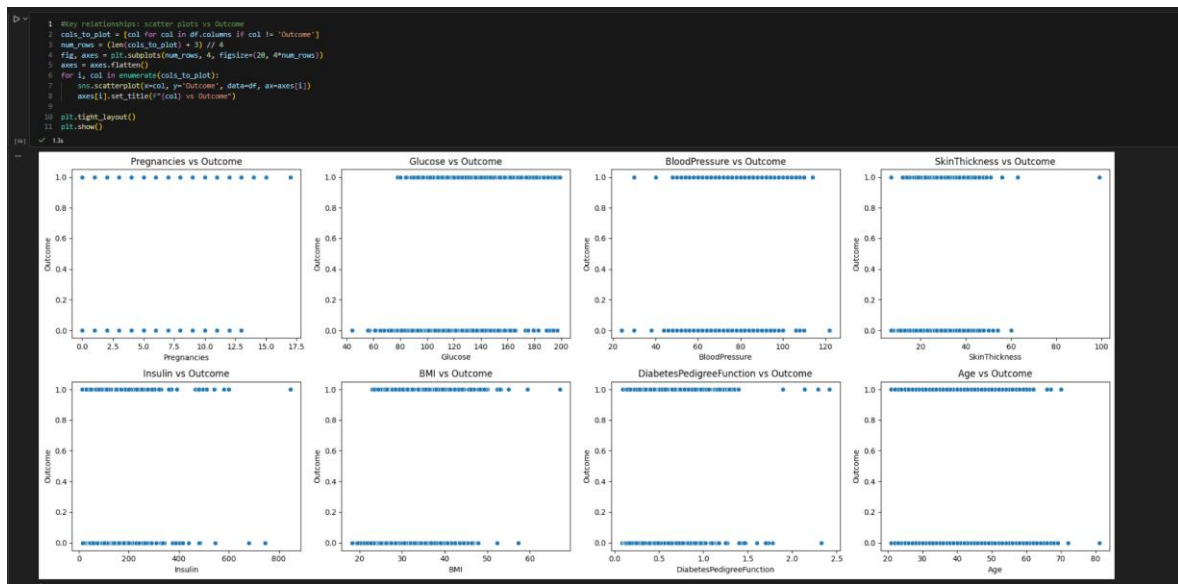


Figure 13 : Scatter plots vs Outcome of All Variables

- Then I created scatter plots between each independent variable and the outcome variable to examine the relationships and identify any potential linear associations. For Further Clarification I have attached only the figure of the Scatterplots separately in the Appendix Section.

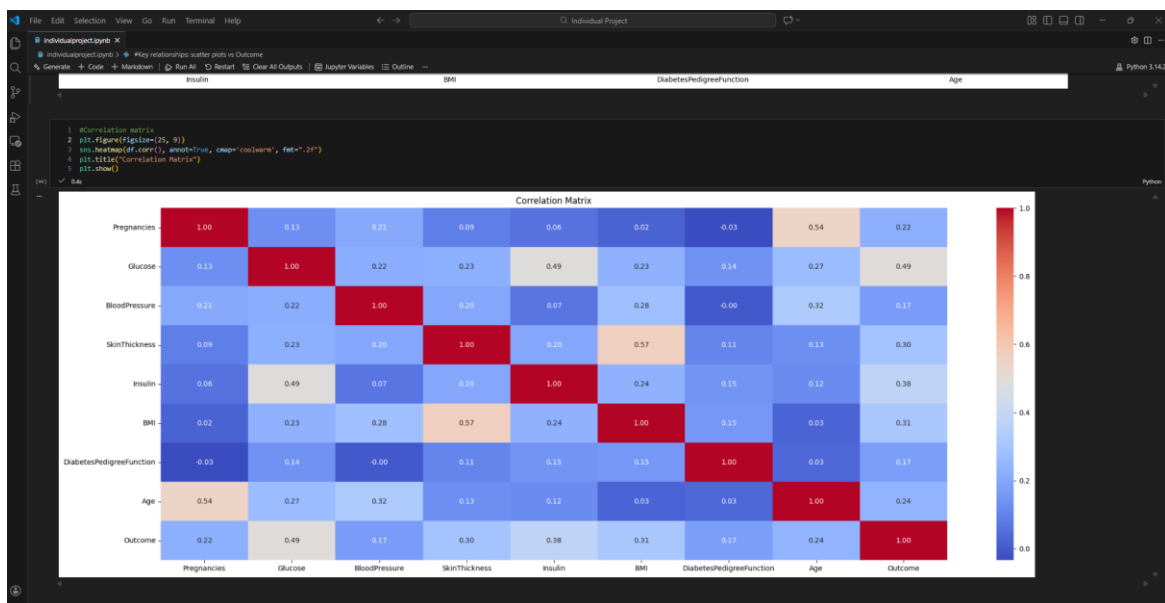


Figure 14 : Correlation Matrix of All Variables

- Finally, I generated a correlation matrix to measure the strength and direction of relationships between variables and detect variables that are too closely related (multicollinearity or high correlation). For further clarification, I have attached only the correlation matrix figure separately in the Appendix Section.

Chapter 4 : Literature Review

Literature Review

Type 2 diabetes mellitus (T2DM) is one of the most common chronic diseases worldwide. Early prediction and effective management are essential to reduce complications and improve patient outcomes. Many recent studies have applied data driven and statistical approaches to predict diabetes risk, manage patients, and guide healthcare decisions. The following review summarizes five important studies in this area.

Xing et al. [1] developed a machine learning model to predict the risk of recurrence after catheter ablation in patients who have T2DM and atrial fibrillation. The study used clinical data and algorithms such as XGBoost to calculate risk scores. While the model performs well for this specific patient group, it may not apply to other diabetic populations. In addition, the study focuses on predicting recurrence rather than new onset diabetes, and it lacks prospective validation. Therefore, the findings are limited to this particular clinical scenario and cannot yet be used for broader population level predictions.

Otok et al. [2] compared two statistical approaches, binary probit regression and Fourier series nonparametric logistic regression to predict diabetes status at Hajj General Hospital in Surabaya. They evaluated the models using accuracy, area under the curve (AUC), sensitivity, and specificity. The study provides strong evidence for model performance, but it relies on traditional statistical techniques and does not use modern machine learning methods. The dataset came from a single hospital, which limits the generalizability of the results. Additionally, the research focused on comparing models and assessing performance, without exploring how these predictions could be applied in everyday clinical practice.

Khan [3] reviewed Bayesian statistical models for predicting T2DM prevalence in urban populations. These models include hierarchical and spatio - temporal approaches, which can capture complex patterns and dependencies in population data. The review synthesizes results from multiple studies but notes that these models are computationally intensive and may be difficult to apply in real - time clinical or public health settings. Evidence from the review may also be affected by differences between the included studies, and there is no clear consensus on standard modeling practices. This highlights the research gap in developing practical, standardized methods for Bayesian prediction in diverse urban populations.

Ahmed et al. [4] focused on predictive modeling for diabetes management in the USA. The study tested several machine learning classifiers, including logistic regression, Random Forest, and support vector

machines, on clinical and demographic datasets. The models achieved good performance metrics, but they were evaluated on a single dataset, limiting confidence in their broader applicability. The technological gap lies in applying these models reliably across different populations. The research also did not explore real - world integration, such as how these predictions could influence patient care or long - term outcomes.

Tanaka et al. [5] examined longitudinal cohort data to predict diabetes onset using multiple machine learning algorithms, including logistic regression, Naive Bayes, XGBoost, and artificial neural networks. Using several models improved predictive accuracy, and the study demonstrates the potential of machine learning for long - term risk prediction. However, external validation in other populations is limited, and practical deployment in healthcare systems remains a challenge. Furthermore, the study emphasizes prediction rather than causal relationships, leaving a gap in understanding how these predictions can guide interventions to prevent diabetes.

Across these studies, several common patterns can be observed. First, quantitative approaches dominate diabetes prediction research, ranging from traditional statistical models to advanced machine learning and Bayesian techniques. Statistical models are easier to interpret but may not capture complex nonlinear relationships, whereas machine learning models can improve predictive performance but often introduce challenges related to transparency and implementation. Second, evidence level gaps are common, as many studies rely on single datasets, retrospective designs, or lack external validation across diverse populations. This limits confidence in whether the models would perform consistently in real-world clinical environments. Technological level gaps are also present, particularly in studies that depend on computationally intensive Bayesian methods or rely solely on traditional statistical techniques without incorporating modern machine learning advancements. Finally, research level gaps highlight the need for broader population coverage, prospective validation, and stronger integration of predictive models into routine healthcare decision-making. Together, these limitations indicate that while predictive modeling for Type 2 diabetes shows strong potential, further work is required to enhance generalizability, practical usability, and real-world clinical impact.

In conclusion, these studies demonstrate the promise of data - driven methods for diabetes analysis and prediction. While statistical and machine learning models provide useful insights, limitations in generalizability, clinical implementation, and standardization remain. Future research should focus on validating predictive models in diverse populations, integrating them into clinical practice, and ensuring they not only predict risk but also support decision - making to improve patient outcomes and public health strategies.

Summary Table

Paper ID	Author/s	Name	Published Year	Methodology (Quantitative / Qualitative)	Gaps		
					Technological Level	Evidence Level	Research Level
S26665 018260 00255 [1]	- Bo Xing - Hongxi Liu - Wanqiu Tao - Tianshu Ren - Qingchun Zhao	Machine Learning -Based Model to Predict Risks of Recurrence after Catheter Ablation in Patients With Type 2 Diabetes Mellitus Combined With Atrial fibrillation	2026	Quantitative - retrospective ML model to predict recurrence risk in diabetic patients.			✓
398074 584[2]	- Bambang W. Otok - Muhammad Zulfadhli - Riwi D. Pangesti - Muhammad I. Kurniawan - Albertus E. P. Haryanto - Darwis - Iwan Kurniawan	Comparison of Binary Probit Regression and Fourier Series Nonparametric Logistic Regression in Modeling Diabetes Status at Hajj General Hospital Surabaya	2025	Quantitative - compared statistical models (probit, logistic, semiparametric) using accuracy and AUC.	✓		
394302 739[3]	Md. Kamrul Khan	Bayesian Statistical Models for Predicting Type 2 Diabetes	2025	Quantitative - reviewed Bayesian models for	✓		

		Prevalence in Urban Populations		diabetes prediction.			
394302 739[4]	• Shahriar Ahmed • Md. Musa Haque • Shah Foysal Hossain • Sarmin Akter • Md. Al Amin • Irin Akter Liza • Ekramul Hasan	Predictive Modeling for Diabetes Management in the USA : A Data - Driven Approach	2024	Quantitative - tested ML classifiers (Logistic Regression, Random Forest, SVM) on patient data.		✓	
S26669 706230 00707[5]	• Marenao Tanaka • Yukinori Akiyama • Kazuma Mori • Itaru Hosaka • Kenichi Kato • Keisuke Endo • Toshifumi Ogawa • Tatsuya Sato • Toru Suzuki • Toshiyuki Yano • Hirofumi Ohnishi • Nagisa Hanawa • Masato Furuhashi	Predictive Modeling for the Development of Diabetes Mellitus Using Key Factors in Various Machine Learning Approaches	2024	Quantitative – Used ML models (Logistic Regression, Naive Bayes, XGBoost, ANN) on cohort data to predict diabetes.		✓	

Table 1 : Literature Review Summary Table

Explanation of Gaps Selected in Summary Table

Paper S2666501826000255 - Research Level Gap

This study developed a machine learning model to predict recurrence after catheter ablation in patients with Type 2 Diabetes and atrial fibrillation. However, it was built only for this specific patient group, making it difficult to apply the findings to other diabetic populations. Since it was not tested on broader groups, the results cannot be fully generalized, creating a Research Level Gap. [1]

Paper 398074584 - Technological Level Gap

The study compared Binary Probit and Fourier Series Logistic Regression models to predict diabetes status. While the statistical analysis was strong, it relied only on traditional methods and did not explore more advanced machine learning techniques. This limitation reflects a Technological Level Gap. [2]

Paper 394302739 - Technological Level Gap

This paper reviewed Bayesian models for predicting Type 2 Diabetes. Although effective, Bayesian methods are often computationally complex and difficult to apply in real-time clinical settings. These practical limitations highlight a Technological Level Gap. [3]

Paper 394302739 - Evidence Level Gap

The researchers tested several machine learning models and reported strong results on their dataset. However, the models were not externally validated or tested in real-world settings. Without broader validation, the study shows an Evidence Level Gap. [4]

Paper S2666970623000707 - Evidence Level Gap

This study applied multiple machine learning approaches using cohort data to predict diabetes. However, the models were not validated across diverse populations. The limited external testing results in an Evidence Level Gap. [5]

Chapter 5 : Research Questions

1. What are the key risk factors contributing to the development of diabetes?

This question aims to identify which variables such as Glucose, Blood Pressure, BMI, Insulin, SkinThickness, and Diabetes Pedigree Function which are most strongly associated with a positive diabetes outcome. Higher glucose levels or BMI might indicate elevated risk, while family history captured by the Diabetes Pedigree Function may reveal genetic predisposition. Studies like Tanaka et al. [5] demonstrate that using multiple risk factors in machine learning models can improve prediction accuracy, while Khan [3] highlights that complex statistical models can capture interactions among risk factors in populations. Ahmed et al. [4] further supports the importance of clinical and demographic variables in identifying patients at higher risk, although broader validation is needed for general use. Understanding these factors can guide early interventions and preventive strategies.

2. Is there a relationship between age and the likelihood of having diabetes?

The Age variable allows exploration of whether diabetes prevalence increases with age or if specific age groups are more vulnerable. Insights from Otok et al. [2] show that statistical models can quantify the impact of patient characteristics, such as age, on diabetes status. Tanaka et al. [5] used longitudinal cohort data to demonstrate that age, combined with other factors, can be a strong predictor of diabetes onset over time. Khan [3] also notes that demographic factors, including age, are important in predicting diabetes prevalence, especially in urban populations. Understanding age - related risk patterns can help target specific groups for screening, lifestyle interventions, and awareness campaigns.

3. How do lifestyle related factors, such as pregnancies or BMI, affect diabetes prevalence?

Variables like Pregnancies and BMI can reflect lifestyle or physiological influences on diabetes risk. For instance, multiple pregnancies or higher BMI may correlate with increased likelihood of diabetes. Ahmed et al. [4] showed that including BMI and other lifestyle variables in predictive models improves accuracy in managing diabetes. Xing et al. [1] highlighted that even in specific patient groups, lifestyle related factors can significantly influence diabetes risk, though broader studies are needed. Tanaka et al. [5] also emphasizes that integrating lifestyle and physiological factors enhances long term prediction and helps inform effective interventions, such as weight management and postpartum care.

Chapter 6 : Modelling and Evaluation

Model Selection

To build a reliable diabetes prediction model, I selected Logistic Regression and Random Forest Classifier as the two ML models for the modelling and evaluation task . These two models work well together because one is simple and easy to explain, while the other is more powerful and better at capturing complex patterns.

1. Logistic Regression

Logistic Regression is a model made for yes or no questions. Since the task here is to predict whether a person has diabetes or not, it is a good fit. One of the biggest advantages of this model is that it is easy to understand. It clearly shows how each health factor such as glucose level, BMI, or age affects the prediction. This is especially important in healthcare, where doctors need to understand why a model makes a certain decision. Instead of acting like a “black box,” Logistic Regression provides clear reasoning.

It also works well with structured data like ours, which includes eight medical measurements. The model trains quickly and makes fast predictions, which is useful in real-world medical situations where time matters. In addition, it gives us a strong baseline model to compare with more advanced methods.

2. Random Forest Classifier

Random Forest is a more advanced machine learning model. It is better at finding complex patterns that simpler models might miss. In real life, health factors often interact with each other. For example, having high glucose and high BMI together might increase diabetes risk more than either factor alone. Random Forest can automatically capture these kinds of interactions without needing extra adjustments.

It also has several practical advantages

- It handles noisy or messy data well.
- It is less likely to overfit because it combines multiple decision trees.

- It can rank features by importance, which will be a greater help in understanding which health factors matter most.

Because medical data can be complex, Random Forest is well suited for improving prediction accuracy.

Why Use Both(2) Models?

Using both models allows me to balance two important goals

- Logistic Regression is simple, clear, and easy to explain.
- Random Forest is more powerful and often more accurate.

By comparing them, I can see whether the simpler model performs well enough or if the more advanced model provides better results. In many cases, both models may perform strongly, allowing me to choose the one that offers the best balance between accuracy and explainability. Additionally, the project guidelines specifically require the use of two different models, which further supports my decision to apply both approaches.

Performance Evaluation Metrics

```
1 # Installing and importing necessary libraries
2 %pip install scikit-learn
3
4 # Importing necessary libraries
5 from sklearn.model_selection import train_test_split
6 from sklearn.linear_model import LogisticRegression
7 from sklearn.ensemble import RandomForestClassifier
8 from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix, classification_report

✓ 24s

Requirement already satisfied: scikit-learn in c:\Users\Dulan Liyanarachchi\AppData\Local\Programs\Python\Python314\Lib\site-packages (1.8.0)
Requirement already satisfied: numpy>=1.24.1 in c:\Users\Dulan Liyanarachchi\AppData\Local\Programs\Python\Python314\Lib\site-packages (from scikit-learn) (2.4.2)
Requirement already satisfied: scipy>=1.10.0 in c:\Users\Dulan Liyanarachchi\AppData\Local\Programs\Python\Python314\Lib\site-packages (from scikit-learn) (1.17.0)
Requirement already satisfied: joblib>=1.3.0 in c:\Users\Dulan Liyanarachchi\AppData\Local\Programs\Python\Python314\Lib\site-packages (from scikit-learn) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.2.0 in c:\Users\Dulan Liyanarachchi\AppData\Local\Programs\Python\Python314\Lib\site-packages (from scikit-learn) (3.6.0)
Note: you may need to restart the kernel to use updated packages.
WARNING: Ignoring invalid distribution ~ip (c:\Users\Dulan Liyanarachchi\AppData\Local\Programs\Python\Python314\Lib\site-packages)
WARNING: Ignoring invalid distribution ~ip (c:\Users\Dulan Liyanarachchi\AppData\Local\Programs\Python\Python314\Lib\site-packages)
WARNING: Ignoring invalid distribution ~ip (c:\Users\Dulan Liyanarachchi\AppData\Local\Programs\Python\Python314\Lib\site-packages)
WARNING: Ignoring invalid distribution ~ip (c:\Users\Dulan Liyanarachchi\AppData\Local\Programs\Python\Python314\Lib\site-packages)
```

Figure 15 : Installing and Importing the required Libraries

- To do the modelling and evaluation I installed scikit - learn library and imported `train_test_split` from `sklearn.model_selection`, `LogisticRegression` from `sklearn.linear_model`, `RandomForestClassifier` from `sklearn.ensemble`, and `accuracy_score`, `precision_score`, `recall_score`, `f1_score`, `confusion_matrix`, `classification_report` from `sklearn.metrics`.

```
1 # Splitting the Data into Training and Testing Data Sets
2 X = df.drop('Outcome', axis=1)
3 y = df['Outcome']
4 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

✓ 0.0s
```

Figure 16 : Splitting the Data into Training and Testing Data Sets

- Then I separated the dataset to x and y variables where x has all inputs for the model and y has the actual output of each input combination(row).
- After that I created the training and testing datasets using `train_test_split` function, x variable, and y variable where 20% of the data from the dataset is allocated to test dataset which will be used for testing purpose of the model.

```

1  # Initializing the Logistic Regression model
2  lr_model = LogisticRegression(random_state=42, max_iter=1000)
3
4  # Training the model
5  lr_model.fit(X_train, y_train)
6
7  # Making predictions using the model
8  lr_pred = lr_model.predict(X_test)
9
10 # Evaluating the Model
11 print("Logistic Regression Model")
12 print("-" * 25, "\n")
13 print(f"Accuracy: {accuracy_score(y_test, lr_pred):.4f}")
14 print(f"Precision: {precision_score(y_test, lr_pred):.4f}")
15 print(f"Recall: {recall_score(y_test, lr_pred):.4f}")
16 print(f"F1 Score: {f1_score(y_test, lr_pred):.4f}")
17 print("\nClassification Report")
18 print(classification_report(y_test, lr_pred))

```

[32] ✓ 0.1s

```

... Logistic Regression Model
-----

Accuracy: 0.7792
Precision: 0.6981
Recall: 0.6727
F1 Score: 0.6852

Classification Report

```

	precision	recall	f1-score	support
0	0.82	0.84	0.83	99
1	0.70	0.67	0.69	55
accuracy			0.78	154
macro avg	0.76	0.76	0.76	154
weighted avg	0.78	0.78	0.78	154

Figure 17 : Logistic Regression model

- Next I trained the Logistic Regression model using the training dataset and used it to make predictions on the test dataset.
- The model achieved about 78% accuracy, correctly predicting most cases.
- For the positive class, it showed around 70% precision and 67% recall, resulting in an F1-score of 69%.

- The negative class was predicted more accurately as it had more samples, but overall, the model captured the patterns in the data well.

```

1  # Initializing the Random Forest model
2  rf_model = RandomForestClassifier(random_state=42, n_estimators=100)
3
4  # Training the model
5  rf_model.fit(X_train, y_train)
6
7  # Making predictions using the model
8  rf_pred = rf_model.predict(X_test)
9
10 # Evaluating the Model
11 print("Random Forest Model")
12 print("-" * 20 , "\n")
13 print(f"Accuracy: {accuracy_score(y_test, rf_pred):.4f}")
14 print(f"Precision: {precision_score(y_test, rf_pred):.4f}")
15 print(f"Recall: {recall_score(y_test, rf_pred):.4f}")
16 print(f"F1 Score: {f1_score(y_test, rf_pred):.4f}")
17 print("\nClassification Report")
18 print(classification_report(y_test, rf_pred))

```

[35] ✓ 0.3s

```

... Random Forest Model
-----

Accuracy: 0.8831
Precision: 0.8136
Recall: 0.8727
F1 Score: 0.8421

Classification Report

```

	precision	recall	f1-score	support
0	0.93	0.89	0.91	99
1	0.81	0.87	0.84	55
accuracy			0.88	154
macro avg	0.87	0.88	0.87	154
weighted avg	0.89	0.88	0.88	154

Figure 18 : Random Forest model

- Then I trained the Random Forest model using the training dataset and used it to make predictions on the test dataset.

- This model also performed quite well, achieving an accuracy of about 88%, correctly predicting most cases.
- For the positive class, the precision was around 81% and recall about 87%, giving an F1-score of 84%.
- The negative class was predicted slightly more accurately, with a precision of 93% and recall of 89%.
- Overall, the model captured the patterns in the data effectively and showed stronger performance compared to simpler models.



Figure 19 : Confusion Matrices

- Then to evaluate model performance, confusion matrices were generated for both the Logistic Regression and Random Forest models.
- These matrices visually represent the number of true positives, true negatives, false positives, and false negatives produced by each model on the test dataset.
- The Logistic Regression model demonstrated balanced performance, correctly classifying the majority of instances. Its confusion matrix showed that most true positives and true negatives were identified accurately, although some misclassifications between the classes were observed.

- The Random Forest model performed slightly better overall, correctly predicting a higher number of cases. Its confusion matrix displayed higher values along the diagonal, indicating more accurate identification of both positive and negative classes.
- By comparing the two confusion matrices side by side using distinct color schemes, it is possible to clearly observe where each model performs well and where it struggles in distinguishing between the two classes, providing an effective visual diagnostic of their prediction behavior.
- Overall, the Random Forest model captured complex patterns in the data more effectively, while the Logistic Regression model served as a simpler yet reasonably reliable baseline for classification.
- For Further Clarification I have attached only the figure of the Confusion Matrices separately in the Appendix Section.

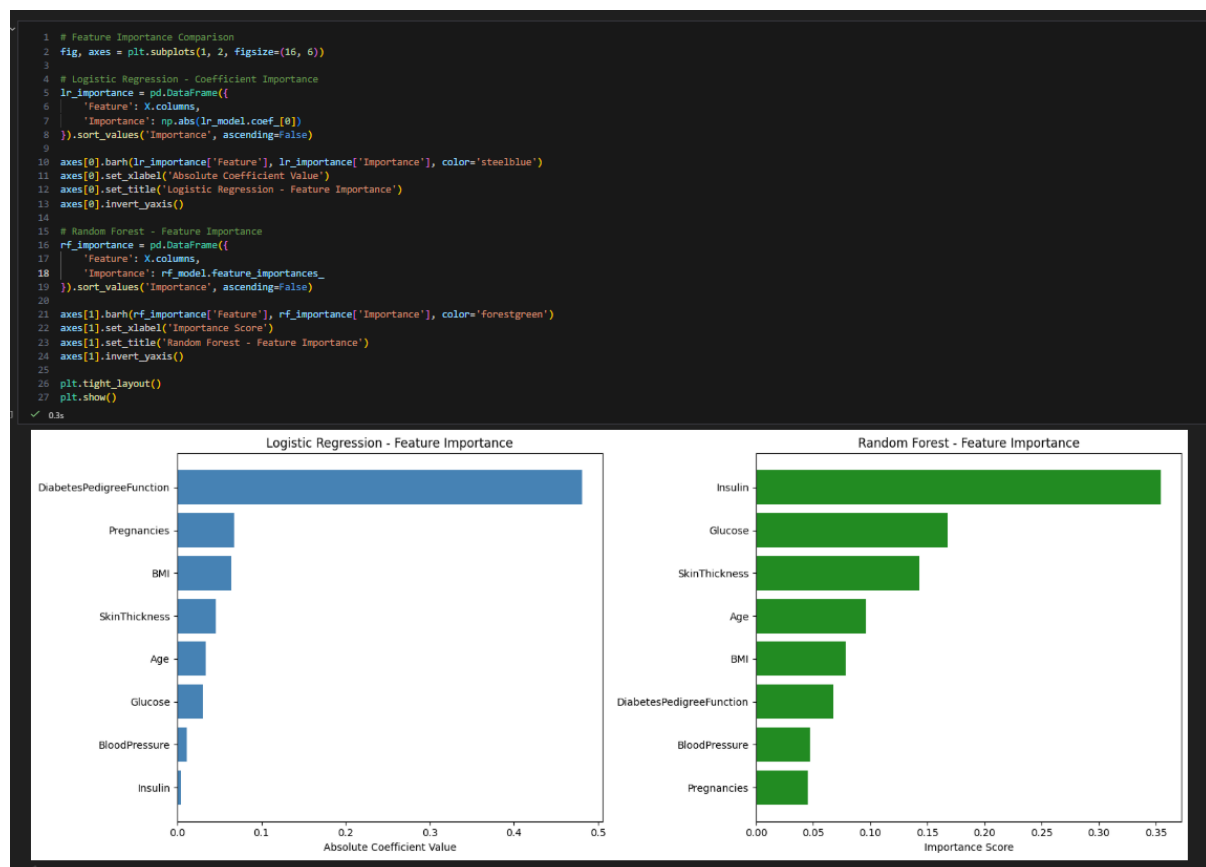


Figure 20 : Feature Importance Comparison

- As the final step, I compared the feature importance between Logistic Regression and Random Forest to see how each model prioritizes variables.
- When I looked at Logistic Regression, I found that DiabetesPedigreeFunction was the most influential feature, followed by Pregnancies, BMI, and SkinThickness, while Insulin and BloodPressure had little impact.

- Then, I examined the Random Forest model, and I noticed that Insulin became the most important feature, with Glucose and SkinThickness also contributing strongly, while DiabetesPedigreeFunction was less significant.
- This difference happened because Logistic Regression captures linear relationships, whereas Random Forest can detect non-linear patterns and interactions.
- I also saw that features like SkinThickness and Age were moderately important in both models, showing some agreement. Overall, by comparing both models, I could identify which features are consistently predictive and which are more model-specific.
- For Further Clarification I have attached only the figure of the Feature Importance Comparison separately in the Appendix Section.

Model Comparison and Selection

I compared Logistic Regression and Random Forest to determine which model is more suitable for predicting diabetes, considering both accuracy and interpretability. Logistic Regression is simple and easy to understand, while Random Forest is more complex but capable of capturing intricate patterns in the data. To compare their performance, I evaluated both models using several key metrics on the test dataset which is shown below in a table clearly.

Performance Evaluation Metric	Logistic Regression Model	Random Forest Model
Accuracy	78%	88%
Precision (Positive Class)	70%	81%
Recall (Positive Class)	67%	87%
F1-Score (Positive Class)	69%	84%
Support (Positive Class)	55	55
Precision (Negative Class)	82%	93%
Recall (Negative Class)	84%	89%
F1-Score (Negative Class)	83%	91%
Support (Negative Class)	99	99

Table 2 : Performance Evaluation Metrics Comparison

The Logistic Regression model achieved an overall accuracy of about 78%. For predicting positive diabetes cases, it showed around 70% precision, 67% recall, and an F1-score of 69%. Negative cases were predicted more accurately, partly due to their higher representation in the dataset. Its confusion matrix indicated most true positives, and true negatives were correctly identified, though some misclassifications occurred. Logistic Regression's simplicity and linear nature make it easy to interpret and explain, establishing it as a reliable baseline. In terms of feature importance, Logistic Regression relied most on DiabetesPedigreeFunction, Pregnancies, BMI, and SkinThickness, reflecting the linear relationships in the data, while features such as Insulin and BloodPressure had little impact.

The Random Forest model outperformed Logistic Regression, with an accuracy of 88%. For positive cases, it achieved 81% precision, 87% recall, and an F1-score of 84%, while negative cases were also predicted more accurately, with 93% precision and 89% recall. Its confusion matrix showed fewer misclassifications, reflecting better overall predictive capability. Random Forest's ability to capture non-linear patterns and feature interactions allowed it to recognize complex relationships that Logistic Regression could not. In this model, Insulin, Glucose, and SkinThickness emerged as the most important features, while DiabetesPedigreeFunction was less significant. Features such as

SkinThickness and Age remained moderately important, indicating some consistency with Logistic Regression but also highlighting the model-specific contributions of certain variables.

Overall, Logistic Regression is simple and interpretable and provides a reliable baseline, but Random Forest demonstrates stronger performance in accuracy and correctly identifying both positive and negative cases. The feature analysis further clarifies why certain variables drive predictions differently depending on the model. While Logistic Regression could be considered if interpretability is a priority, I found Random Forest to be the more effective model for predicting diabetes due to its higher accuracy and ability to capture complex patterns in the data.

Summary and Conclusion

Findings Summary and Conclusion

This study focused on predicting diabetes risk using machine learning techniques and a dataset containing clinical and demographic information. Through data exploration and analysis, several important health factors were identified as strong indicators of diabetes. In particular, glucose level, Body Mass Index (BMI), age, blood pressure, and diabetes pedigree function showed clear relationships with diabetes outcomes. These findings suggest that both physiological and genetic factors play a major role in increasing a person's risk of developing diabetes.

Data preprocessing was an important step in ensuring accurate results. Some variables contained zero values that were not medically realistic, so they were treated as missing data and replaced using median values. Outliers were kept because extreme health measurements can reflect real medical conditions rather than errors. These steps helped improve the overall quality of the dataset and supported better model performance.

Two machine learning models, Logistic Regression and Random Forest, were developed to answer the research questions. Logistic Regression provided a simple and interpretable baseline model with an accuracy of around 78%, while the Random Forest model achieved approximately 88% accuracy with higher precision, recall, and F1 scores.

Comparing feature importance across the two models provided additional insights into diabetes risk factors. Logistic Regression highlighted DiabetesPedigreeFunction, Pregnancies, BMI, and SkinThickness as the most influential predictors, reflecting linear relationships in the data. In contrast, Random Forest identified Insulin, Glucose, and SkinThickness as the strongest contributors, capturing non-linear interactions that Logistic Regression could not. Features like SkinThickness and Age were moderately important in both models, indicating some consistent predictors. This comparison allowed identification of features that are robust across models versus those whose impact is model-specific, enhancing the interpretability and clinical relevance of the findings. (The feature importance comparison figure is provided in the Appendix.)

Overall, the study confirms that machine learning can effectively identify individuals at higher risk of diabetes based on clinical measurements and that more complex models like Random Forest can capture intricate patterns in the data, improving predictive performance.

Answers for Research Questions

1. What are the key risk factors contributing to the development of diabetes?
 - Glucose, Insulin, BMI, Age, SkinThickness, and DiabetesPedigreeFunction were identified as the most important predictors.
 - Both models confirmed that glucose and insulin levels play a major role in diabetes prediction.
 - Genetic influence (DiabetesPedigreeFunction) and physiological factors such as BMI also significantly increased risk.
2. Is there a relationship between age and the likelihood of having diabetes?
 - Yes. Age showed moderate importance in both models.
 - The analysis suggests that diabetes likelihood increases with age when combined with other risk factors.
 - This supports literature findings such as Md. Kamrul Khan [3], which emphasize demographic influence in diabetes prevalence prediction.
3. How do physiological and lifestyle-related variables, such as pregnancies or BMI, affect diabetes prevalence?
 - BMI and Pregnancies were significant contributors, particularly in Logistic Regression.
 - Higher BMI was associated with higher diabetes risk.
 - These findings are supported by Shahriar Ahmed et al. [4], who demonstrated that, including clinical and demographic variables improves prediction accuracy.

Implications

The findings of this study are valuable for healthcare professionals and policymakers. By recognizing key risk factors early, healthcare providers can encourage preventive actions such as maintaining a healthy weight, monitoring glucose levels, and promoting lifestyle changes. Early detection can help delay or prevent the development of diabetes, ultimately reducing long-term health complications.

Additionally, the strong performance of the Random Forest model highlights the potential of using machine learning in clinical decision-making. Predictive tools based on these methods could support doctors in screening patients more accurately and allocating healthcare resources more efficiently. The approach used in this study could also be applied to predict other chronic diseases, making it useful beyond diabetes research.

Limitations

Despite its contributions, this study has several limitations. The dataset represents a specific population, which means the results may not fully apply to other groups with different lifestyles or genetic backgrounds. Some important factors such as diet, physical activity, medication use, and detailed family history were not included in the dataset, which may affect prediction accuracy.

The dataset was also cross-sectional, capturing information at a single point in time. Therefore, the study can identify associations but cannot establish cause-and-effect relationships. Although Random Forest produced higher accuracy, it is more complex than Logistic Regression and may be harder to interpret in real clinical settings where understanding the reasoning behind predictions is crucial.

Future Research Directions

Future research should focus on using larger and more diverse datasets to improve the reliability and generalizability of the findings. Including additional variables such as lifestyle habits, genetic information, and socioeconomic factors could lead to more accurate predictions.

Researchers could also test more advanced algorithms like Gradient Boosting or neural networks to further enhance model performance. Simultaneously, using interpretability techniques such as SHAP or LIME would help explain how models make predictions, increasing trust among healthcare professionals.

Finally, validating these models with data from different populations and studying their real-world impact in healthcare settings will be important. This would help determine whether machine learning models can truly improve early detection, support preventive care, and contribute to better health outcomes for individuals at risk of diabetes.

References

[1]

B. Xing, H. Liu, W. Tao, T. Ren, and Q. Zhao, "Machine Learning-Based Model to Predict Risks of Recurrence after Catheter Ablation in Patients With Type 2 Diabetes Mellitus Combined With Atrial fibrillation," *Heart Rhythm O2*, Jan. 2026, doi: <https://doi.org/10.1016/j.hroo.2026.01.013>.

[2]

B. W. Otok *et al.*, "COMPARISON OF BINARY PROBIT REGRESSION AND FOURIER SERIES NONPARAMETRIC LOGISTIC REGRESSION IN MODELING DIABETES STATUS AT HAJJ GENERAL HOSPITAL SURABAYA," *BAREKENG: Jurnal Ilmu Matematika dan Terapan*, vol. 20, no. 1, pp. 0255–0270, Nov. 2025, doi: <https://doi.org/10.30598/barekengvol20iss1pp0255-0270>.

[3]

Md. Kamrul. Khan, "BAYESIAN STATISTICAL MODELS FOR PREDICTING TYPE 2 DIABETES PREVALENCE IN URBAN POPULATIONS," vol. 04, no. 02, pp. 370–406, Jun. 2025, doi: <https://doi.org/10.63125/db2e5054>.

[4]

N. S. Ahmed *et al.*, "Predictive Modeling for Diabetes Management in the USA: A Data-Driven Approach," *Journal of Medical and Health Studies*, vol. 5, no. 4, pp. 214–228, Dec. 2024, doi: <https://doi.org/10.32996/jmhs.2024.5.4.24>.

[5]

M. Tanaka *et al.*, "Predictive modeling for the development of diabetes mellitus using key factors in various machine learning approaches," *Diabetes Epidemiology and Management*, p. 100191, Dec. 2023, doi: <https://doi.org/10.1016/j.deman.2023.100191>

Appendix

Chapter 3

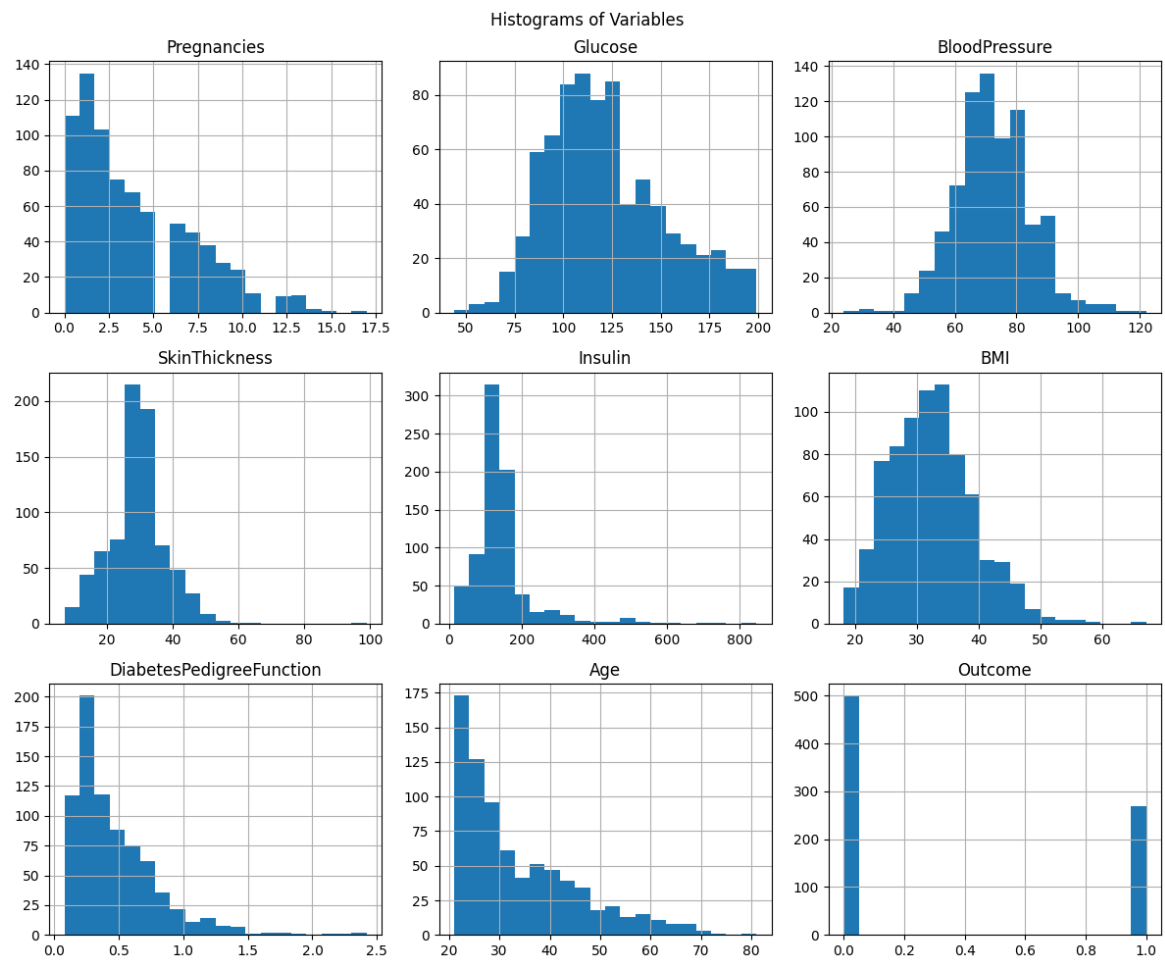


Figure 21 : Histogram Distributions Figure

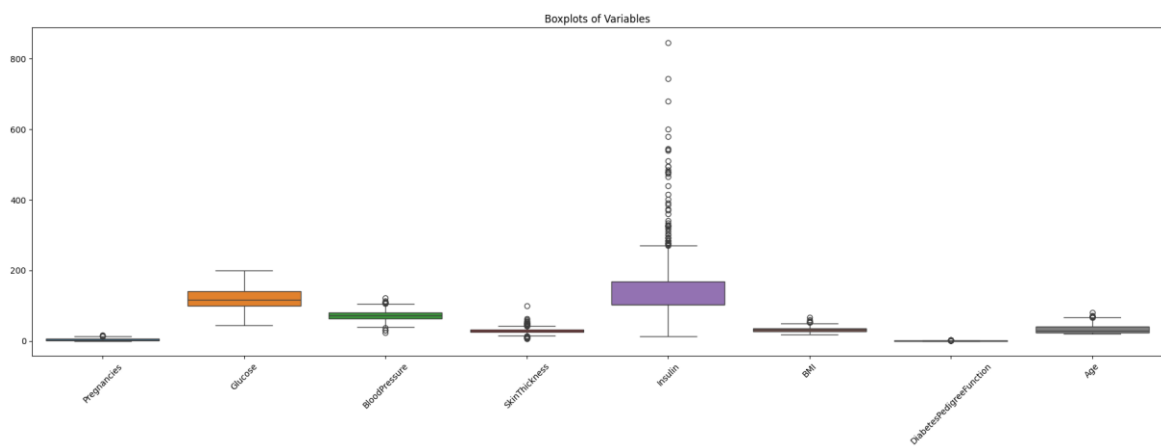


Figure 22 : Boxplot Distributions Figure

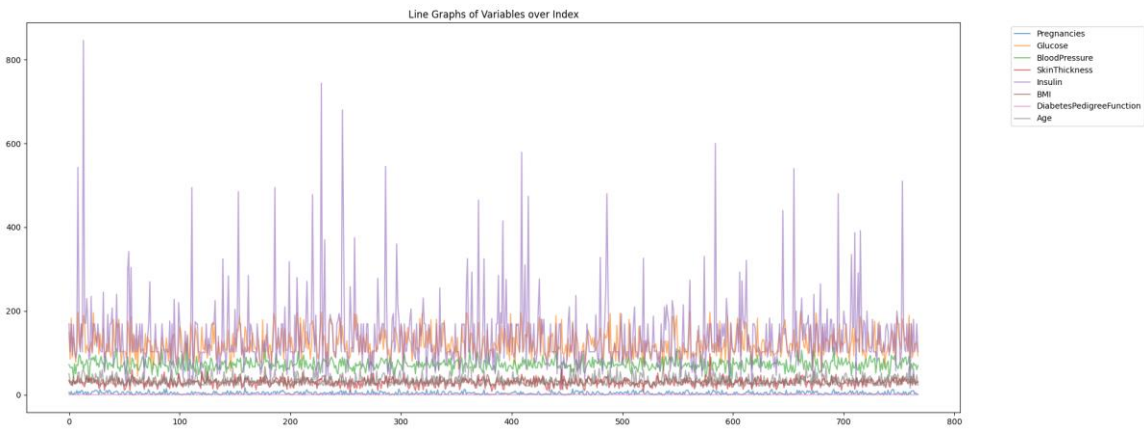


Figure 23 : Line Graphs Figure

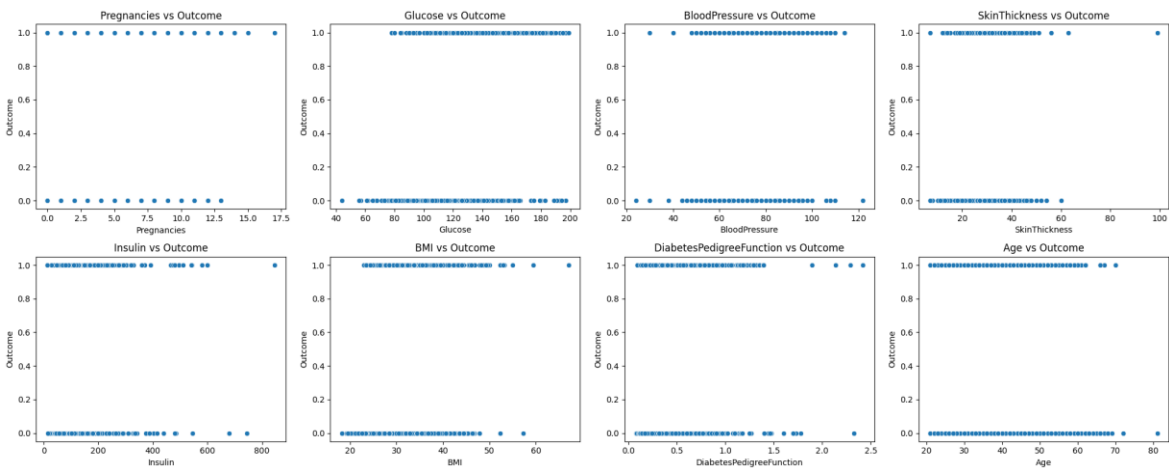


Figure 24 : Scatter plots Figure

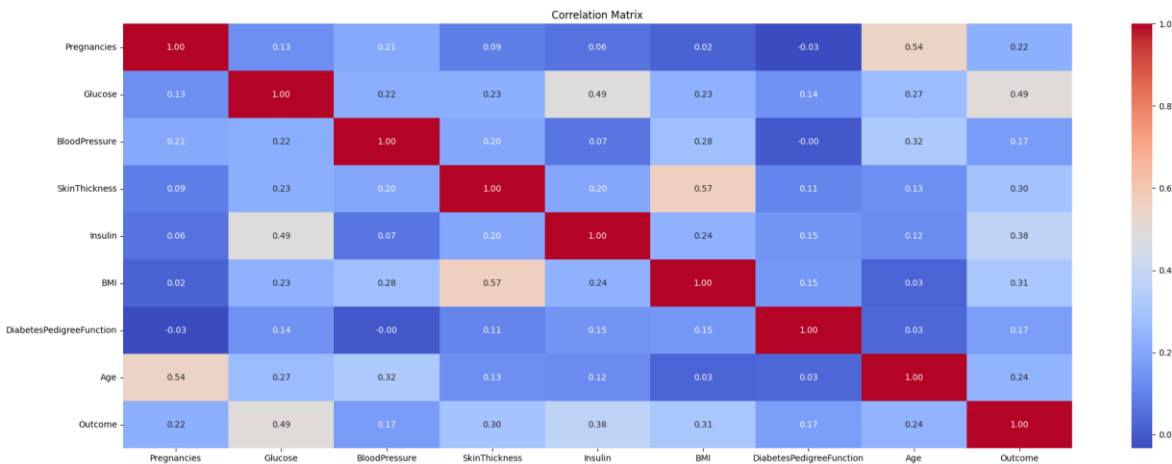


Figure 25 : Correlation Matrix Figure

Chapter 6

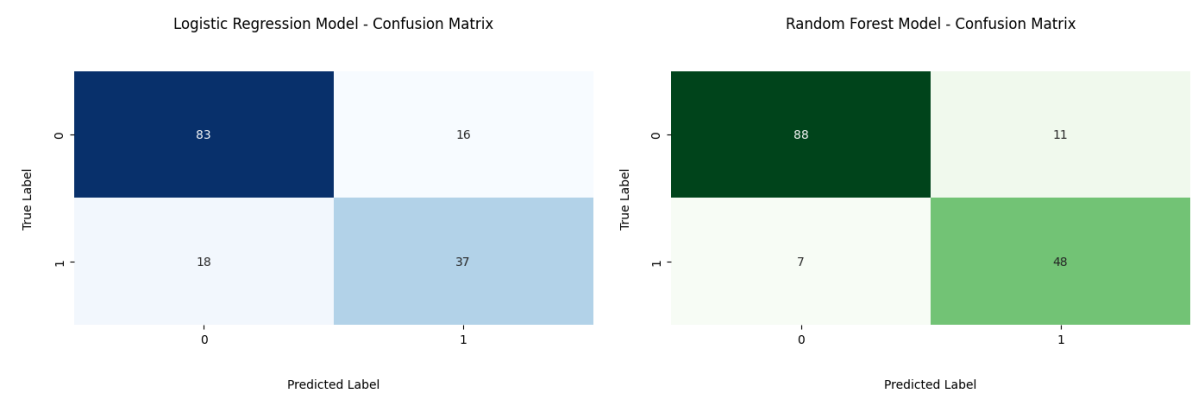


Figure 26 : Confusion Matrices Figure

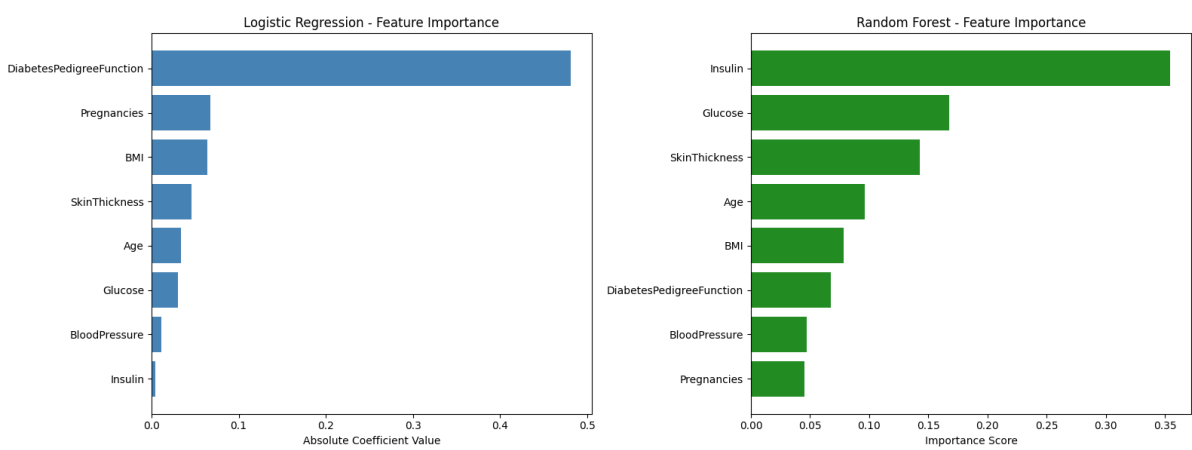


Figure 27 : Feature Importance Comparison